

Chapter 2

LITERATURE REVIEW

2.1 Introduction

This chapter reviews scholarly and standards-based literature relevant to the Marrybrown Sales and Payment Analytics Platform as a company-owned reporting platform for an externally managed point-of-sale (POS) reporting environment. The review is structured to establish the theoretical and practical basis for the architecture and validation approach used in this project. Fundamental concepts are introduced first, followed by related studies and comparative discussion, workflow and interface considerations, and a brief discussion of the technology choices aligned to the literature.

2.2 Fundamental Theory and Concepts

2.2.1 Sales and Payment Reporting in Vendor-Managed POS Environments

In vendor-managed POS environments, transaction processing and report access may be delivered through the same externally managed service boundary. From a cloud-computing perspective, this arrangement places operational reporting within a third-party availability, security, and change-management context rather than wholly under organisational control (Mell & Grance, 2011; Badger et al., 2012; Jansen & Grance, 2011). For sales and payment reporting, this matters because daily totals, payment breakdowns, and reconciliation outputs are often consumed through vendor-managed portal interfaces and exports rather than through an internally operated reporting stack.

The literature also highlights lock-in effects in cloud adoption, where switching providers or rebuilding capabilities can incur high technical and operational costs (Armbrust et al., 2010). These costs are amplified when report logic exists primarily in a vendor-managed portal and is not fully documented or reproducible by the organisation. Consequently, continuity-oriented initiatives commonly prioritise retaining organisational access to usable transaction data and establishing reporting capabilities that are less exposed to vendor-side service disruptions (Badger et al., 2012; Armbrust et al., 2010).

2.2.2 Data Warehousing and Replication-Based Analytical Stores

Data warehousing literature frames a data warehouse as an integrated repository designed to support reporting and analysis rather than operational transaction processing (Inmon, 2005; Kimball & Ross, 2013). For sales and payment analytics platforms, this repository forms the database foundation for storing duplicated transactional data in a form that remains queryable for reporting while preserving a clear relationship to operational records. Traditional designs frequently apply modelling and transformation choices (e.g., dimensional modelling) to optimise analytical queries, but such transformations can increase complexity and may obscure traceability to source records if governance is weak.

For reporting continuity, an alternative design is a fidelity-first replicated analytical store that preserves the structure and content of source transactional tables. This approach prioritises traceability and simplifies reconciliation by reducing ambiguity about how report outputs relate to replicated source records. In practice, the choice between an optimised dimensional model and a replicated fidelity-first store depends on the primary objective: performance optimisation versus assured traceability and parity with externally generated reports (Inmon, 2005; Kimball & Ross, 2013).

2.2.3 Data Integration Pipelines: ETL vs ELT and Idempotent Loads

Data integration pipelines are commonly described using ETL (extract-transform-load), where data is transformed prior to loading into the target repository (Kimball & Caserta, 2004). In sales and payment analytics platforms, these pipelines form the process layer that moves vendor-originated data into the organisation-controlled reporting environment. Contemporary data platform practices also adopt ELT patterns, in which raw data is loaded first and transformations are applied in the target environment. This aligns with schema-on-read approaches that preserve raw data while enabling flexible downstream transformations (Azzabi et al., 2024).

For operational reliability, pipeline design must consider rerun safety and operational recovery. Idempotent loading patterns reduce the risk of duplicate records and support repeatable refresh cycles when failures occur or when late data corrections are detected. In practice, idempotency is achieved through controlled load boundaries (e.g., date-range refresh) and deterministic processing rules that yield the same replicated state for the same boundary.

2.2.4 Replication, Availability, and Consistency Considerations

Replication is used to improve availability and support resilience by maintaining copies of data across systems. In distributed and replicated systems, trade-offs between consistency, availability, and operational complexity are central design considerations (Kleppmann, 2017). For reporting continuity platforms, strict real-time consistency is typically not required; instead, the practical objective is timely access to sufficiently recent and internally consistent data, supported by a refresh process and validation evidence that stakeholders can trust during reporting-intensive periods.

Accordingly, replication design in reporting contexts is commonly paired with operational controls (e.g., rerun capability, logging, and post-load checks) to manage failure recovery and to provide confidence in the replicated state. These controls

support both day-to-day reporting and the investigative workflows that arise when discrepancies are detected.

2.2.5 Schema-on-Read, Database Design, Semantic Layers, and Service Interfaces

Schema-on-read practices emphasise preserving raw or lightly transformed datasets and applying transformations at consumption time, which supports iterative refinement of business logic while maintaining traceability (Azzabi et al., 2024). For reporting platforms that duplicate operational sales and payment data, this principle supports database designs in which replicated tables remain close to source structure, while report-specific calculations are handled separately for auditability and change control (Inmon, 2005; Kimball & Ross, 2013).

Data repository design and semantic design are therefore related but distinct concerns. Data warehousing literature differentiates between storage structures used to retain and organise data and presentation or consumption structures used to deliver consistent analytical views (Inmon, 2005; Kimball & Ross, 2013). In this project context, reconstructed report logic is treated as semantic-layer logic: duplicated transactional data are retained for traceability, while business rules, derived fields, and report-specific aggregation behaviour are centralised above the database layer to reduce metric drift and simplify maintenance.

For service delivery, REST is a widely cited architectural style for network-based systems, emphasising stateless communication, uniform interfaces, and resource-oriented design (Fielding, 2000). A RESTful API therefore provides a standard mechanism to deliver report outputs to a portal while decoupling report consumption from underlying database implementation details and keeping business-rule changes versioned and auditable at the service layer.

2.2.6 Reverse Engineering and Black-Box Validation of Legacy Reports

When vendor report logic is proprietary and not fully documented, reverse engineering and design recovery provide a structured lens for inferring behaviour from observable artefacts (Chikofsky & Cross, 1990). In such settings, black-box testing methods support parity validation by comparing system outputs against vendor exports under identical parameters (Myers et al., 2011). For reporting platforms, this process extends beyond matching totals; it also requires validating calculation rules and edge-case handling (e.g., returns, voids, split tenders, and rounding behaviour) to reduce operational reconciliation risk.

2.2.7 Data Quality and Reconciliation for Financial Reporting

Data quality is commonly framed as fitness for use, where suitability depends on the consumer's needs and the use context (Wang & Strong, 1996). Standards and reference models describe core quality characteristics such as accuracy, completeness, consistency, and timeliness (International Organization for Standardization, 2008). In sales and payment reporting, these characteristics are directly relevant because stakeholders require totals, breakdowns, and reconciliation outputs that are dependable for decision-making and audit preparation.

Data quality management literature discusses practical techniques for assessing and improving data quality, including profiling and validation rules (Batini & Scannapieco, 2006). In replicated reporting contexts, these techniques are operationalised through reconciliation checks (e.g., cross-report consistency of totals) and repeatable parity comparisons against vendor exports for agreed validation windows.

2.2.8 Iterative Development Approaches for Evolving Requirements

Iterative development approaches emphasise incremental delivery and feedback-driven refinement (Beck et al., 2001). Scrum formalises iterative planning and review through time-boxed cycles and continuous refinement (Schwaber & Sutherland, 2020). In software engineering practice, iterative approaches are commonly recommended when requirements are uncertain or when continuous validation is required to manage risk (Pressman & Maxim, 2014; Sommerville, 2015). In report reconstruction projects, requirements and edge cases often emerge during parity validation, which makes iterative delivery suitable for progressively stabilising business rules while retaining validation evidence.

2.3 Related Studies/Systems and Comparative Discussion

Related literature on vendor-managed reporting environments, organisation-controlled reporting repositories, and schema-on-read analytical platforms provides a comparative basis for the architecture adopted in this project. Cloud computing literature recognises vendor dependency and lock-in as concerns that can affect cost, portability, governance, and operational control (Armbrust et al., 2010; Badger et al., 2012). Data warehousing literature explains how organisation-controlled repositories can improve reporting reliability and analytical access (Inmon, 2005; Kimball & Ross, 2013). Survey work on data lakes and schema-on-read practices further highlights the value of preserving raw data and applying transformations flexibly at the consumption layer (Azzabi et al., 2024).

However, fewer sources provide detailed guidance on reconstructing proprietary vendor reporting logic while demonstrating report-level parity through defensible validation artefacts. This creates a practical gap for organisations that require continuity reporting but must operate without full access to vendor calculation definitions.

Table 2.1 provides a comparative discussion between a vendor-managed reporting portal and a company-owned reporting platform, highlighting the trade-offs that motivate the architecture adopted in this project.

Criterion	Vendor-managed portal only	Company-owned reporting platform
Availability and continuity	Reporting access depends on vendor service availability and change management (Badger et al., 2012; Jansen & Grance, 2011).	Provides an alternative reporting path by operating on replicated data under organisational control (Kleppmann, 2017).
Data transparency and traceability	Users typically interact through portal views and exports; traceability to raw transactions may be limited by vendor interface constraints.	Fidelity-first replication supports traceability from report outputs to replicated source records (Inmon, 2005).
Report logic control	Report definitions and changes are vendor-controlled; organisational adaptation is constrained.	Business rules are reconstructed and versioned in a semantic layer, enabling auditable updates.
Validation burden	Vendor portal is treated as the reference, but internal validation is limited to exported outputs.	Requires black-box parity validation and reconciliation evidence to build trust in reconstructed logic (Myers et al., 2011; Wang & Strong, 1996).
Cost and complexity	Lower internal build complexity, but potential lock-in and dependency costs (Armbrust et al., 2010).	Higher engineering and validation effort initially; benefits depend on sustained operational use and governance.

Table 2.1: Comparative discussion of vendor-managed reporting versus company-owned reporting

Data warehousing literature, together with research on schema-on-read practices, also motivates a design trade-off between transformation-heavy analytical models and replication-first designs. Dimensional modelling supports analytical performance and standardised business views, whereas replication-first designs prioritise traceability and parity alignment to source representations (Inmon, 2005; Kimball & Ross, 2013). For continuity-oriented reporting where parity and reconciliation are primary goals, the architecture used in this project adopts replication for fidelity and applies transformations at the semantic layer, consistent with schema-on-read principles (Azzabi et al., 2024).

Based on this synthesis, the project architecture combines a fidelity-first replicated datastore to support traceability and reconciliation, an ELT workflow with idempotent refresh to support safe reruns and late adjustments, a semantic/API layer for centralised business rules and service delivery, reverse engineering with black-box parity validation to infer and validate proprietary behaviour, and iterative development practices to manage emerging edge cases (Azzabi et al., 2024; Beck et al., 2001; Chikofsky & Cross, 1990; Kleppmann, 2017; Myers et al., 2011; Wang & Strong, 1996).

2.4 Workflow and Interface Considerations for Operational Reporting

Portals

In operational sales and payment reporting, interaction is typically organised around a report-retrieval workflow rather than around dashboard-style monitoring. Enterprise reporting systems commonly present users with a catalogue of available reports, after which the user selects a report, specifies the required parameters, executes the query, reviews the returned output, and exports the result if needed. Oracle MICROS Reporting and Analytics reflects this pattern through report-list navigation, prompt-based parameter entry, filtering, drill-down behaviour, and export functions for generated reports (Oracle, 2023). A comparable workflow is also evident in paginated reporting systems, where users select a report, enter parameters before rendering the output, and export the result in formats such as Excel or PDF (Microsoft,

2026). For this type of use, the display purpose is primarily detailed lookup and operational reporting, which makes tabular presentation especially appropriate when users need precise values for verification, reconciliation, and follow-up action (Few, 2012).

Usability literature on self-service portals further suggests that routine operational tasks benefit from clear navigation, understandable prompts, consistent interaction patterns, and reduced effort in repeated use (Matloobtalab & Ferati, 2025). In addition, Shneiderman's information-seeking framework highlights the practical importance of filtering and details-on-demand when users must narrow a large report space or inspect a result set progressively according to immediate task needs (Shneiderman, 1996). Within a reporting portal, these considerations translate into structured report selection, predictable parameter controls, and result views that support progressive inspection without unnecessary visual or procedural complexity.

Taken together, these sources suggest that a sales and payment reporting portal should prioritise report discoverability, parameter-driven retrieval, readable tabular outputs, and export support for reconciliation-oriented work. This interpretation is consistent with the practical workflow observed in vendor-managed reporting environments and provides a conceptual basis for the portal requirements and interface decisions discussed in Chapter 4. Table 2.2 summarises the reporting portal design considerations most relevant to the implemented platform.

Design consideration	Supporting source(s)	Relevance to the implemented platform
Report catalogue and navigation	Oracle (2023); Matloobtalab & Ferati (2025)	Users should be able to locate relevant reports efficiently from a structured list or menu.
Parameter specification before query execution	Oracle (2023)	Report retrieval should support explicit parameter entry such as outlet, date range, and status before output generation.

Readable tabular output for operational checking	Few (2012)	Sales and payment reporting often requires exact values for checking, reconciliation, and follow-up action rather than visual summaries alone.
Filtering and progressive inspection of results	Shneiderman (1996); Oracle (2023)	Users should be able to narrow report outputs and inspect relevant details without unnecessary interface complexity.
Consistency of prompts, labels, and interaction flow	Matloobtalab & Ferati (2025)	Consistent controls across reports reduce learning effort and improve usability for recurring operational tasks.
Export support for reconciliation and offline review	Oracle (2023); Microsoft (2026)	Retrieved report outputs should be exportable to support reconciliation, sharing, and further offline review.

Table 2.2: Reporting portal workflow and interface considerations for the implemented platform

2.5 Technology Selection Considerations

Within the implemented platform, Microsoft SQL Server, a Python-based ELT workflow, FastAPI, and React were selected as engineering technologies that fit the literature-grounded architectural direction established in this chapter. SQL Server supports the company-managed reporting repository, the Python-based ELT workflow supports controlled extraction and refresh, FastAPI supports RESTful service delivery for reconstructed reports, and React supports parameter-driven retrieval, tabular viewing, and export workflows for internal users (FastAPI, n.d.; React, n.d.; Fielding, 2000). This section is included to connect the literature-grounded architecture to the actual technology stack adopted in the project, while detailed implementation discussion is reserved for Chapters 4 and 5.

2.6 Synthesis and Rationale Map

Table 2.3 synthesises how the literature reviewed in this chapter supports the architectural and methodological choices adopted in this project. The synthesis links each major design or methodological choice to its underlying academic rationale, thereby making the basis for the implemented platform explicit and traceable.

Method/design choice	Rationale grounded in literature	Key sources
Iterative development for report reconstruction	Requirements and edge cases emerge during parity validation; iterative delivery supports risk reduction through frequent feedback and refinement.	Beck et al. (2001); Schwaber & Sutherland (2020); Pressman & Maxim (2014); Sommerville (2015)
ELT workflow with idempotent refresh	Loading raw data first supports schema-on-read transformations and iterative rule refinement; idempotent boundaries support reruns and recovery.	Kimball & Caserta (2004); Azzabi et al. (2024)
Fidelity-first replication for traceability	Preserving source structure supports reconciliation and explainability of outputs, which is central in financial reporting contexts.	Inmon (2005); Kleppmann (2017)
Consistency/availability framing for reporting	Reporting prioritises timely, consistent snapshots rather than strict real-time consistency; operational controls improve reliability.	Kleppmann (2017)
Schema-on-read with semantic layer exposed via API	Separating duplicated storage structures from business-rule and service-interface logic	Inmon (2005); Kimball & Ross (2013); Azzabi et al.

	preserves traceability, reduces metric drift, and supports auditable evolution.	(2024); Fielding (2000)
Reverse engineering + black-box parity validation	Proprietary vendor logic requires inference from observable behaviour; black-box testing provides empirical parity evidence.	Chikofsky & Cross (1990); Myers et al. (2011)
Data quality and reconciliation checks	Fitness-for-use for financial reporting depends on accuracy, completeness, and consistency; reconciliation operationalises quality assurance.	Wang & Strong (1996); International Organization for Standardization (2008); Batini & Scannapieco (2006)
Cloud dependency risk and lock-in considerations	Vendor dependence can introduce availability/security/change-management risks and lock-in costs; continuity designs mitigate exposure.	Mell & Grance (2011); Badger et al. (2012); Jansen & Grance (2011); Armbrust et al. (2010)

Table 2.3: Synthesis / rationale map linking adopted choices to literature

2.7 Summary

This chapter reviewed key concepts and prior work relevant to vendor-managed sales and payment reporting environments, replication-based analytical stores, ELT workflows, schema-on-read and database design, semantic layers, workflow and interface considerations for operational reporting portals, reverse engineering and black-box validation, data quality and reconciliation, and iterative development. Taken together, the literature supports an internal reporting platform that prioritises data fidelity, reconciliation, controlled report delivery, and iterative refinement. The next chapter presents the methodology used in this project to implement and validate that approach.